

MySQL Essentials for Big Data Engineering

From SELECT to Window Functions — the SQL foundation every data engineer needs, plus CSV imports and hands-on practice tasks.

MUST KNOW

GOOD TO KNOW

OPTIONAL

CORE WORKFLOW

Contents

- 1. SELECT — Retrieving Data
- 2. WHERE — Filtering Rows
- 3. GROUP BY — Aggregating Rows
- 4. HAVING — Filtering Groups
- 5. ORDER BY — Sorting Results
- 6. Aggregate Functions
- 7. JOINS
- 8. Window Functions
- 9. CASE WHEN — Conditional Logic
- 10. CTE — Common Table Expressions
- 11. Subqueries
- 12. Basic DDL — Defining Structure
- 13. Importing a CSV File into MySQL
- 13b. Importing a CSV — Command Line & GUI Alternatives
- 14. UNION — Combining Result Sets
- 15. String Functions
- 16. Date Functions
- 17. NULL-Handling Functions
- 18. Views
- 19. UPDATE — Modifying Existing Rows
- 20. DELETE — Removing Rows
- 21. Stored Procedures
- 22. Triggers
- 23. Transactions
- 24. Indexes
- 25. Normalization

Plus: Practice Tasks (intermediate level) at the end of the document.

1. SELECT — Retrieving Data

MUST KNOW

SELECT is the entry point of almost every query you will ever write. It defines which columns come back from a table, in what order, and under what alias. In a warehouse, you rarely SELECT * in production code — you name columns explicitly so schema changes upstream don't silently break downstream jobs.

```
-- Pull specific columns from the products dimension, renaming for clarity
SELECT
  product_id,
  product_name,
  unit_price AS price_usd -- alias: renames the output column
FROM dim_products
LIMIT 10; -- cap the result set while exploring data
```

Big Data angle: This is the SQL equivalent of `df.select('product_id', 'product_name', F.col('unit_price').alias('price_usd')).limit(10)` in PySpark — same intent, different syntax.

2. WHERE — Filtering Rows

MUST KNOW

WHERE filters rows before any grouping happens. It runs on raw, row-level data, so it can't reference an aggregate like COUNT() or SUM() directly — that's what HAVING is for (section 4). Filters here are pushed down as early as possible for performance.

```
-- Sales above 100 EUR, placed in the last 90 days
SELECT sale_id, customer_id, total_amount, sale_date
FROM fact_sales
WHERE total_amount > 100
AND sale_date >= '2025-05-01' -- combine conditions with AND / OR
AND customer_id IS NOT NULL; -- exclude anonymous/incomplete rows
```

Big Data angle: This maps directly to Spark's predicate pushdown: filters written this early in a query (or a `.filter()` call) let the engine skip reading irrelevant partitions entirely, instead of loading everything and filtering in memory.

3. GROUP BY — Aggregating Rows

MUST KNOW

GROUP BY collapses many rows into one row per unique value (or combination of values) in the grouping columns. Every column in SELECT that isn't inside an aggregate function must appear in GROUP BY — this is the single most common syntax error beginners hit.

```
-- Total revenue and order count per customer
SELECT
customer_id,
COUNT(*) AS order_count, -- rows per group
SUM(total_amount) AS total_revenue -- sum within each group
FROM fact_sales
GROUP BY customer_id; -- one output row per customer_id
```

Big Data angle: This is conceptually identical to `df.groupBy('customer_id').agg(count('*'), sum('total_amount'))` in PySpark — GROUP BY is the SQL `groupBy+agg` pair.

4. HAVING — Filtering Groups

MUST KNOW

HAVING filters AFTER aggregation, which is why it can reference SUM(), COUNT(), AVG() and friends, while WHERE cannot. A useful mental model: WHERE trims rows, HAVING trims groups.

```
-- Only customers who generated more than 500 in total revenue
SELECT
customer_id,
SUM(total_amount) AS total_revenue
FROM fact_sales
GROUP BY customer_id
HAVING SUM(total_amount) > 500; -- filters the aggregated result, not raw rows
```

Big Data angle: Filtering after a `groupBy/agg` is more expensive than filtering before it — apply a WHERE first whenever the condition doesn't depend on the aggregate, exactly as you'd order `.filter()` before `.groupBy()` in a Spark pipeline when possible.

5. ORDER BY — Sorting Results

MUST KNOW

ORDER BY sorts the final result set. It runs last in logical query order (after SELECT, WHERE, GROUP BY, HAVING), so you can sort by a column alias defined in SELECT. Default direction is ascending (ASC); use DESC for descending.

```
-- Top spenders first
SELECT customer_id, SUM(total_amount) AS total_revenue
FROM fact_sales
GROUP BY customer_id
ORDER BY total_revenue DESC -- highest revenue first
LIMIT 5; -- top 5 customers only
```

Big Data angle: Sorting is a full shuffle in a distributed engine like Spark (`orderBy` triggers a global sort across partitions) — in MySQL on a single node it's cheap by comparison, but the logical concept of 'sort last' still holds in both worlds.

6. Aggregate Functions

MUST KNOW

COUNT, SUM, AVG, MIN and MAX summarize a set of rows into a single value. They can be used with GROUP BY for per-group summaries, or alone for a single grand total across the whole table. COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column only — a subtle but interview-relevant distinction.

```
-- Category-level summary: revenue, average order value, and item count
SELECT
  p.category,
  COUNT(*) AS num_orders,
  SUM(f.total_amount) AS revenue,
  AVG(f.total_amount) AS avg_order_value,
  MIN(f.total_amount) AS smallest_order,
  MAX(f.total_amount) AS largest_order
FROM fact_sales f
JOIN dim_products p ON f.product_id = p.product_id
GROUP BY p.category;
```

7. JOINS

MUST KNOW

JOINS combine rows from two or more tables using a matching key. INNER JOIN keeps only matching rows on both sides; LEFT JOIN keeps every row from the left table and fills unmatched right-side columns with NULL. This is the mechanism that turns a normalized star schema back into a flat, analysis-ready result.

```
-- Flatten the star schema: one row per sale with customer & product detail
SELECT
  f.sale_id,
  c.customer_name,
  p.product_name,
  p.category,
  f.quantity,
  f.total_amount
FROM fact_sales f
INNER JOIN dim_customers c ON f.customer_id = c.customer_id -- must match on both
sides
LEFT JOIN dim_products p ON f.product_id = p.product_id; -- keep sale even if
-- product lookup is missing
```

Big Data angle: This is exactly what a Spark broadcast join does when one side of the join (a small dimension table like dim_products) is small enough to fit in memory on every executor — same relational logic, distributed execution.

8. Window Functions

MUST KNOW

Window functions compute a value across a set of related rows (the 'window') WITHOUT collapsing them into one row, unlike GROUP BY. PARTITION BY defines the window per group, and ORDER BY inside OVER() defines the row ordering used for ranking or running calculations.

```
-- Rank each customer's orders by size, and compare to their previous order
SELECT
  customer_id,
  sale_date,
  total_amount,
  RANK() OVER (
    PARTITION BY customer_id -- reset ranking per customer
    ORDER BY total_amount DESC -- biggest order = rank 1
  ) AS order_rank,
  LAG(total_amount) OVER (
    PARTITION BY customer_id
    ORDER BY sale_date
  ) AS previous_order_amount -- value from the prior row in this window
FROM fact_sales;
```

Big Data angle: PARTITION BY / ORDER BY inside OVER() is line-for-line the same idea as pyspark.sql.Window.partitionBy().orderBy() — if you already use rowsBetween, dense_rank, or row_number in PySpark, this syntax will feel immediately familiar.

9. CASE WHEN — Conditional Logic

MUST KNOW

CASE WHEN is SQL's if/elif/else. It evaluates conditions top to bottom and returns the value from the first match, falling back to ELSE (or NULL if ELSE is omitted). It's commonly used to bucket continuous values into labeled tiers.

```
-- Bucket each sale into a size tier
SELECT
  sale_id,
  total_amount,
  CASE
    WHEN total_amount >= 500 THEN 'Large'
    WHEN total_amount >= 100 THEN 'Medium'
    ELSE 'Small'
  END AS order_size_tier -- first matching WHEN wins, top to bottom
FROM fact_sales;
```

Big Data angle: Equivalent to `F.when(cond1, 'Large').when(cond2, 'Medium').otherwise('Small')` in PySpark — a pattern you'll recognize from feature engineering and RFM-style customer segmentation.

10. CTE — Common Table Expressions

MUST KNOW

A CTE (WITH clause) names a temporary result set that the rest of the query can reference, as if it were a table. It exists only for the duration of that query. CTEs make multi-step logic readable and let you reuse the same intermediate result more than once without repeating the subquery.

```
-- Step 1: compute revenue per customer. Step 2: filter to top spenders only.
WITH customer_revenue AS (
  SELECT customer_id, SUM(total_amount) AS total_revenue
  FROM fact_sales
  GROUP BY customer_id
)
SELECT customer_id, total_revenue
FROM customer_revenue -- reference the CTE like a normal table
WHERE total_revenue > 1000
ORDER BY total_revenue DESC;
```

Big Data angle: A CTE is the SQL cousin of assigning an intermediate DataFrame to a variable in PySpark (`df_revenue = df.groupBy(...).agg(...)`) before running a second transformation on top of it — same 'name the intermediate step' habit.

11. Subqueries

MUST KNOW

A subquery is a query nested inside another query — in the WHERE clause, the FROM clause, or SELECT. Unlike a CTE, it isn't named and typically isn't reused. Correlated subqueries (referencing the outer query) run once per outer row, so they're worth watching for performance; uncorrelated ones run once total.

```
-- Customers whose average order is above the global average order value
SELECT customer_id, customer_name
FROM dim_customers
WHERE customer_id IN (
  SELECT customer_id -- subquery runs first, produces a list of IDs
  FROM fact_sales
  GROUP BY customer_id
  HAVING AVG(total_amount) > (
    SELECT AVG(total_amount) FROM fact_sales -- global average, one value
  )
);
```

12. Basic DDL — Defining Structure

MUST KNOW

DDL (Data Definition Language) — CREATE, ALTER, DROP — defines the shape of your database rather than its data. This is the schema every example in this document is built on: two dimension tables and one fact table linked by foreign keys.

```
-- The star-schema warehouse used in every example below
CREATE TABLE dim_customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  city VARCHAR(50),
  country VARCHAR(50),
  signup_date DATE
);

CREATE TABLE dim_products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category VARCHAR(50),
  unit_price DECIMAL(10,2)
);

CREATE TABLE fact_sales (
  sale_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT,
  product_id INT,
  sale_date DATE,
  quantity INT,
  total_amount DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES dim_customers(customer_id),
  FOREIGN KEY (product_id) REFERENCES dim_products(product_id)
);
```


Big Data angle: This is the relational mirror of a PySpark StructType schema definition, or a Hive DDL script — the same bronze/silver/gold discipline applies: define types and keys deliberately instead of letting a loader infer them.

13. Importing a CSV File into MySQL

CORE WORKFLOW

Getting raw data INTO MySQL is a prerequisite most tutorials skip. There are three practical routes: **LOAD DATA INFILE** (fastest, pure SQL), the **mysqlimport** command-line utility (a thin wrapper around LOAD DATA), and the **MySQL Workbench Table Data Import Wizard** (GUI, best for one-off exploration). For anything repeatable in a pipeline, LOAD DATA INFILE is the one worth mastering.

The workflow is always: (1) create the target table with the right column types, (2) run the import, (3) verify row counts and spot-check a few rows.

```
-- 1) Create a table that matches the CSV's columns and types
CREATE TABLE stg_customers_csv (
  customer_id INT,
  customer_name VARCHAR(100),
  city VARCHAR(50),
  country VARCHAR(50),
  signup_date DATE
);

-- 2) Load the file. LOCAL means the file lives on the CLIENT machine
-- (omit LOCAL if the .csv already sits on the MySQL server itself)
LOAD DATA LOCAL INFILE '/path/to/customers.csv'
INTO TABLE stg_customers_csv
FIELDS TERMINATED BY ',' -- comma-separated values
OPTIONALLY ENCLOSED BY '"' -- handles quoted fields like "Doe, John"
LINES TERMINATED BY '\n'
IGNORE 1 ROWS -- skip the header row
(customer_id, customer_name, city, country, signup_date);

-- 3) Sanity check: did every row make it in?
SELECT COUNT(*) FROM stg_customers_csv;
```

Big Data angle: Loading raw CSVs into a small staging table before reshaping them is the same bronze-layer idea from medallion architecture — land the data as-is first, validate it, then transform into silver/gold. In practice you'd often stage through MySQL only briefly before the real transform happens in Spark.

13b. Importing a CSV — Command Line & GUI Alternatives

CORE WORKFLOW

If you'd rather not write LOAD DATA by hand, two equivalent options exist. Both ultimately call the same import mechanism under the hood.

```
# Option A -- mysqlimport command-line utility (filename must match table name)
mysqlimport --local --fields-terminated-by=',' --ignore-lines=1 \
-u root -p my_database /path/to/dim_customers.csv

# Option B -- MySQL Workbench GUI
# Right-click a schema -> Table Data Import Wizard -> choose the .csv file
# -> map CSV columns to table columns -> Next -> Import
# Best for a quick one-off load; not scriptable, so avoid it in production ETL.
```

14. UNION — Combining Result Sets

GOOD TO KNOW

UNION stacks the results of two or more SELECT statements that share the same number and type of columns. Plain UNION removes duplicate rows across the combined set (an implicit DISTINCT); UNION ALL keeps every row, including duplicates, and is noticeably faster since no de-duplication pass is needed.

```
-- Combine two years' worth of archived sales tables into one result
SELECT sale_id, customer_id, total_amount, sale_date FROM fact_sales_2024
UNION ALL -- keep all rows, including duplicates
SELECT sale_id, customer_id, total_amount, sale_date FROM fact_sales_2025;
```

Big Data angle: Directly analogous to `df1.union(df2)` in PySpark — and just like Spark, prefer UNION ALL unless you specifically need de-duplication, since removing duplicates forces an expensive sort/hash step over the whole combined set.

15. String Functions

GOOD TO KNOW

String functions clean and reshape text data — a constant need with real-world, messy source data. CONCAT joins values, TRIM removes stray whitespace, UPPER/LOWER normalize case, and SUBSTRING extracts part of a string.

```
SELECT
CONCAT(customer_name, '(', country, ')') AS display_name,
UPPER(country) AS country_code,
TRIM(city) AS clean_city,
SUBSTRING(customer_name, 1, 1) AS initial
FROM dim_customers;
```

16. Date Functions

GOOD TO KNOW

Date functions extract parts of a date, compute differences, and shift dates forward or backward — essential for any time-based reporting (monthly revenue, cohort age, days-since-last-order).

```
SELECT
sale_id,
YEAR(sale_date) AS sale_year,
MONTH(sale_date) AS sale_month,
DATEDIFF(CURDATE(), sale_date) AS days_since_sale,
DATE_ADD(sale_date, INTERVAL 30 DAY) AS follow_up_date
FROM fact_sales;
```

Big Data angle: The same operations you'd reach for with `pyspark.sql.functions.year()`, `datediff()`, and `date_add()` — the naming conventions are close enough that moving between MySQL and Spark SQL rarely requires relearning this piece.

17. NULL-Handling Functions

GOOD TO KNOW

NULL means 'unknown/missing', not zero or empty string — and it doesn't behave like a normal value in comparisons (NULL = NULL is NULL, not TRUE). COALESCE returns the first non-NULL value in a list; IFNULL is a two-argument shortcut; NULLIF returns NULL if two values are equal (handy for avoiding divide-by-zero).

```
SELECT
customer_id,
COALESCE(city, country, 'Unknown') AS best_available_location,
IFNULL(total_amount, 0) AS safe_amount,
total_amount / NULLIF(quantity, 0) AS unit_price_safe -- avoids /0 error
FROM fact_sales;
```

18. Views

GOOD TO KNOW

A VIEW is a saved SELECT query that behaves like a virtual, always-up-to-date table. It doesn't store data itself — it re-runs the underlying query each time you SELECT from it. Views are useful for hiding join complexity behind a simple, reusable name.

```
CREATE VIEW vw_customer_revenue AS
SELECT
c.customer_id,
c.customer_name,
SUM(f.total_amount) AS total_revenue
FROM dim_customers c
JOIN fact_sales f ON c.customer_id = f.customer_id
GROUP BY c.customer_id, c.customer_name;

-- Now query it exactly like a table
SELECT * FROM vw_customer_revenue WHERE total_revenue > 1000;
```

19. UPDATE — Modifying Existing Rows

OPTIONAL

UPDATE changes existing rows in place. Always pair it with a WHERE clause — an UPDATE without WHERE rewrites every single row in the table.

```
UPDATE dim_products
SET unit_price = unit_price * 1.05 -- 5% price increase
WHERE category = 'Electronics'; -- never omit WHERE on an UPDATE
```

20. DELETE — Removing Rows

OPTIONAL

DELETE removes rows matching a condition. Like UPDATE, a missing WHERE clause deletes every row. For clearing an entire table quickly, TRUNCATE TABLE is faster than DELETE (but can't be rolled back inside a transaction in the same way).

```
DELETE FROM fact_sales
WHERE sale_date < '2020-01-01'; -- remove sales older than the retention window
```

21. Stored Procedures

OPTIONAL

A stored procedure bundles a sequence of SQL statements under a name you can CALL later, optionally with parameters. Useful for encapsulating repeatable business logic directly inside the database.

```
DELIMITER //
CREATE PROCEDURE get_customer_orders(IN p_customer_id INT)
BEGIN
SELECT sale_id, sale_date, total_amount
FROM fact_sales
WHERE customer_id = p_customer_id
ORDER BY sale_date DESC;
END //
DELIMITER ;

CALL get_customer_orders(101);
```

22. Triggers

OPTIONAL

A trigger is a stored procedure that fires automatically BEFORE or AFTER an INSERT, UPDATE, or DELETE on a specific table — commonly used for audit trails or keeping a derived value in sync.

```

DELIMITER //
CREATE TRIGGER trg_log_price_change
BEFORE UPDATE ON dim_products
FOR EACH ROW
BEGIN
IF NEW.unit_price <> OLD.unit_price THEN
INSERT INTO price_change_log (product_id, old_price, new_price, changed_at)
VALUES (OLD.product_id, OLD.unit_price, NEW.unit_price, NOW());
END IF;
END //
DELIMITER ;

```

23. Transactions

OPTIONAL

A transaction groups multiple statements into a single all-or-nothing unit: COMMIT saves all changes, ROLLBACK undoes all of them. This protects against partial writes if something fails halfway through — critical when a single logical operation touches more than one table.

```

START TRANSACTION;

UPDATE dim_products SET unit_price = unit_price - 10 WHERE product_id = 5;
INSERT INTO fact_sales (customer_id, product_id, sale_date, quantity, total_amount)
VALUES (101, 5, CURDATE(), 2, 180.00);

COMMIT; -- both statements succeed together, or use ROLLBACK; to undo both

```

24. Indexes

OPTIONAL

An index is a lookup structure built on one or more columns that lets MySQL find matching rows without scanning the whole table — the single biggest lever for query performance on large tables, at the cost of slightly slower writes.

```
CREATE INDEX idx_fact_sales_customer ON fact_sales (customer_id);

-- EXPLAIN shows whether MySQL actually uses the index for a given query
EXPLAIN SELECT * FROM fact_sales WHERE customer_id = 101;
```

Big Data angle: Conceptually close to Spark's partitioning/bucketing and predicate pushdown on partitioned Parquet output — both are ways of avoiding a full scan by narrowing down where the data you need actually lives.

25. Normalization

OPTIONAL

Normalization organizes tables to minimize redundant data, typically summarized as 1NF/2NF/3NF: atomic column values, no partial dependency on part of a key, and no column depending on another non-key column. The star schema used throughout this document is a deliberately DE-normalized design for analytics — understanding normalization is what lets you explain why that trade-off is made.

```
-- Un-normalized: repeats customer_name and country on every sale row
-- sale_id | customer_name | country | product | amount

-- Normalized (this document's schema): split into fact + dimensions
-- fact_sales(sale_id, customer_id, product_id, amount)
-- dim_customers(customer_id, customer_name, country)
-- dim_products(product_id, product_name)
```

Big Data angle: Warehouses intentionally denormalize (star/snowflake schemas) to minimize joins at query time — OLTP systems normalize to minimize update anomalies. Knowing both lets you explain, in an interview, why a fact/dimension model looks the way it does.

Practice. Practice Tasks — Intermediate Level

CORE WORKFLOW

Every task below uses the same three tables from this document: **fact_sales**, **dim_customers**, **dim_products**. Work through them roughly in order — later groups assume you're comfortable with the earlier ones.

Filtering, Grouping & Aggregation

- 1 Find the top 5 product categories by total revenue, sorted highest first.
- 2 List every customer who placed at least 3 orders AND whose average order value exceeds 150.
- 3 Return the number of distinct customers who bought something in each calendar month of the current year.

JOINS

- 1 Write a query that lists customers who have NEVER placed an order (hint: LEFT JOIN + IS NULL on the fact table's key).
- 2 Produce one row per sale showing customer_name, product_name, category, and total_amount, ordered by total_amount descending.

Window Functions

- 1 For each customer, rank their orders from largest to smallest using DENSE_RANK, and return only rank 1 (their single biggest order).
- 2 Compute a running (cumulative) total of revenue per customer, ordered by sale_date, using SUM() OVER (... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).
- 3 Using LAG(), flag any sale where total_amount dropped more than 50% compared to that same customer's previous order.

CTEs & Subqueries

- 1 Rewrite the 'customers above average order value' subquery from Section 11 as a CTE instead, and compare readability.
- 2 Using a CTE, build an RFM-style segmentation: Recency (days since last order), Frequency (order count), Monetary (total spend) — one row per customer.

CASE WHEN, UNION & NULL Handling

- 1 Add a CASE WHEN column that labels each customer as 'New' (signup_date within 90 days) or 'Returning'.
- 2 Simulate two regional sales tables and combine them with UNION ALL, then verify no rows were unintentionally dropped versus UNION.
- 3 Using COALESCE, produce a clean customer contact list that falls back from city to country to 'Unknown' when a location field is missing.

DDL & CSV Import

- 1 Design and CREATE a staging table for a CSV export of daily sales, then write the LOAD DATA INFILE statement to populate it, skipping the header row.
- 2 After importing, write a validation query that compares row counts between the CSV staging table and the final fact_sales table to catch load errors.

Optional / Stretch

- 1 Write a stored procedure that accepts a category name and returns total revenue for that category.
- 2 Wrap an UPDATE (price change) and its corresponding INSERT (audit log row) in a single transaction with COMMIT/ROLLBACK.
- 3 Add an index to support the customer-lookup query from the JOINS section, then use EXPLAIN to confirm MySQL is using it.